Modeling Human Activity States Using Hidden Markov Models

Human Activity Recognition with Smartphone Sensor Data

Authors: Wilsons Navid Wado Tiwa & Didier Ganza

GitHub: https://github.com/Wilsons-Navid/-hidden-markov-model-activity-recognition

1. Background and Motivation

Smartphones come with accelerometers and gyroscopes that constantly produce motion data, but the actual activity behind those signals is never directly observed. Someone walking and someone standing both generate accelerometer readings, just different ones. Hidden Markov Models work well here because they are built for exactly this kind of problem: inferring a sequence of hidden states (the activities) from noisy observations (the sensor data). In this project, we built a Gaussian HMM from scratch in NumPy to classify four activities (Standing, Walking, Jumping, and Still) using data we collected from two different Android phones running the **Sensor Logger app**. Both phones sampled at the same rate, which let us test whether the model could generalize across different hardware without resampling.

2. Data Collection and Preprocessing

2.1 Data Collection

Two group members each collected data on a different Android phone using the Sensor Logger app. We recorded four activities in 5-10 second clips: Standing (phone held at waist level), Walking (at a steady pace), Jumping (continuous), and Still (phone on a flat surface). Both the accelerometer (x, y, z in m/s^2) and gyroscope (x, y, z in rad/s) were recorded. We ended up with 57 recordings total.

Phone	Device	Sampling Rate	Date	Files
Phone 1	STK-L22	20ms (50 Hz)	2026-03-04	28
Phone 2	Infinix X6855	20ms (50 Hz)	2026-02-28	29

Table 1: Data collection summary. Both phones use the same 20ms sampling rate, so no resampling or harmonization was needed.

Activity	Files	Phone 1	Phone 2	Total Duration
Standing	15	7	8	132.2s (2.2 min)
Walking	14	7	7	117.6s (2.0 min)
Jumping	14	7	7	121.8s (2.0 min)
Still	14	7	7	112.8s (1.9 min)

Table 2: Dataset breakdown per activity. All activities exceed the minimum 1 minute 30 seconds requirement.

2.2 Preprocessing

Sensor Logger saves each recording as a zip with separate accelerometer and gyroscope CSVs. We extracted these, merged them by nearest timestamp into one CSV per recording (7 columns: seconds_elapsed, accel_x, accel_y, accel_z, gyro_x, gyro_y, gyro_z), and named them with the pattern activity_phoneN_NN.csv (e.g., walking_phone1_03.csv).

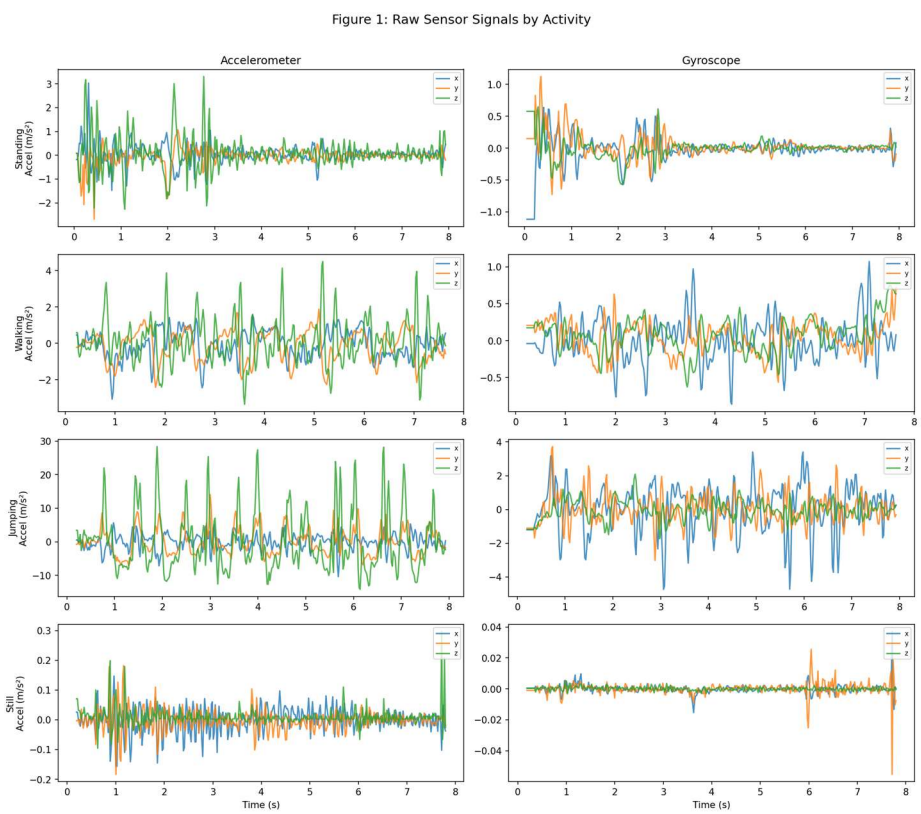

Figure 1: Raw accelerometer (left) and gyroscope (right) signals for one recording of each activity. Standing and Still are mostly flat. Walking has a rhythmic pattern, and Jumping shows large spikes.

2.3 Windowing

We split the continuous signals into windows of 50 samples each, which at 50 Hz gives 1-second windows. We used 50% overlap (step size of 25 samples). One second is long enough to capture a full walking gait cycle (~0.5-1.0s) and at least one jump, and it gives us 1 Hz resolution for the FFT. The overlap roughly doubles our training samples while keeping temporal context. This gave us 663 windows across all 57 files.

2.4 Feature Extraction

We pulled 28 features from each 1-second window, split between time-domain and frequency-domain:

Feature	Count	Domain	Justification
Mean (per axis)	6	Time	Baseline signal level; Standing has distinct z-accel from gravity
Std Deviation (per axis)	6	Time	Movement intensity; Jumping has high std, Still near-zero
RMS (per axis)	6	Time	Overall signal energy; higher for dynamic activities
Signal Magnitude Area	1	Time	Total body acceleration across all axes
Inter-axis Correlation	3	Time	Walking shows correlated axes from rhythmic motion
Dominant Frequency (per accel axis)	3	Frequency	Walking peaks at ~2 Hz; Still has no peak
Spectral Energy (per accel axis)	3	Frequency	Separates high-energy Jumping from low-energy Still

Table 3: Feature summary. We normalized all 28 features with Z-score standardization (zero mean, unit variance) so that features with different units and scales contribute equally.

3. HMM Setup and Implementation

3.1 Model Components

We wrote a Gaussian HMM from scratch in NumPy. The components are:

Component	Symbol	Description
Hidden States	Z	4 activities: Standing, Walking, Jumping, Still

Observations	X	28-dimensional feature vectors from windowed sensor data
Transition Matrix	A (4x4)	Probability of transitioning between activities
Emission Parameters	B = {mu_k, sigma_k}	Gaussian mean and diagonal covariance per state
Initial Probabilities	pi (4x1)	Probability of starting in each activity

Table 4: HMM model components. We used diagonal covariance matrices to keep things computationally stable and avoid singularity problems given our relatively limited training data.

3.2 Baum-Welch Training

We trained the model with Baum-Welch (EM) on 49 training sequences, one per file. We initialized emission parameters from the ground-truth labels, which helped convergence. In the E-step, the Forward and Backward algorithms (run in log space to avoid underflow) compute posterior state probabilities (gamma) and pairwise transition posteriors (xi). The M-step then updates pi, A, means, and covariances to increase data likelihood. We stopped when the change in log-likelihood dropped below 1e-4. Training converged in 6 iterations, with log-likelihood going from 4,509 to 7,682.

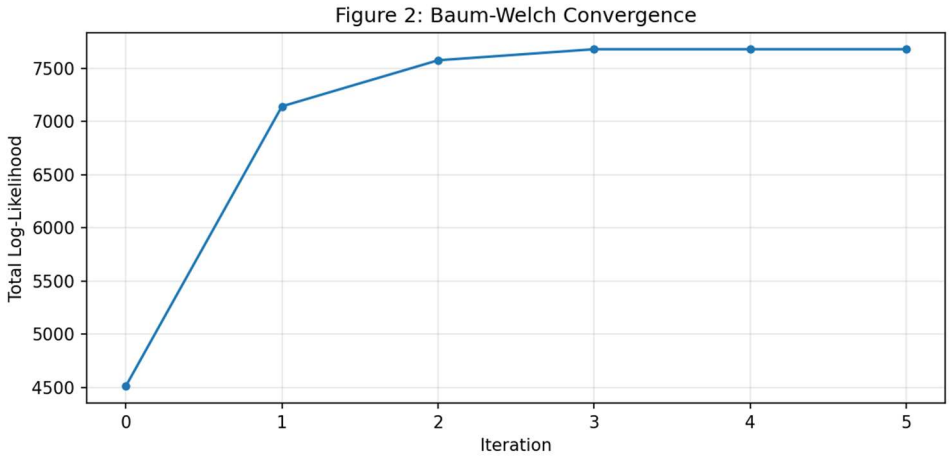

Figure 2: Baum-Welch convergence. Log-likelihood increases monotonically and levels off after 6 iterations.

3.3 Viterbi Decoding

We implemented Viterbi decoding to find the most probable state sequence for a given observation sequence. It uses dynamic programming in log space to trace the highest-probability path through the state trellis, then backtracks to recover the state sequence. We use it both to verify training and to predict on test data.

4. Results and Interpretation

4.1 Learned Transition Probabilities

The learned transition matrix (Figure 3) has high values along the diagonal, close to 1.0. This makes sense: if someone is walking, they are probably still walking one second later. The off-diagonal values are small but nonzero, so the model can still handle real transitions between activities without jumping states on noisy windows.

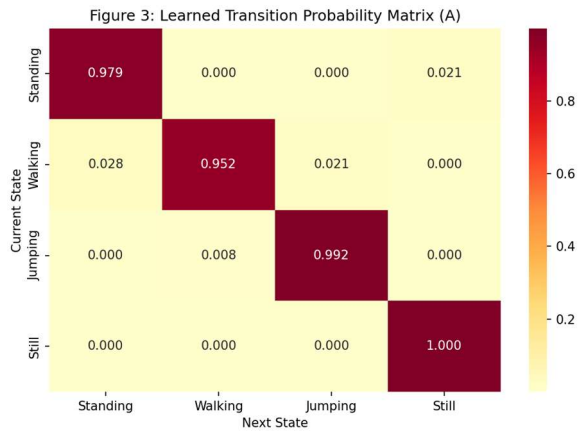

Figure 3: Learned transition probability matrix. The high diagonal values match what we would expect from real activity patterns.

4.2 Emission Distributions

The emission heatmaps (Figure 4) show that the four activities occupy fairly different regions of feature space. Jumping has the highest means for acceleration standard deviation, SMA, and spectral energy, which is expected given how much movement it involves. Still has the lowest values on most features. Standing and Still have similar mean accelerometer values, which explains why the model occasionally confuses them. Jumping also has the highest variance in its emissions, since jump signals are less regular than other activities.

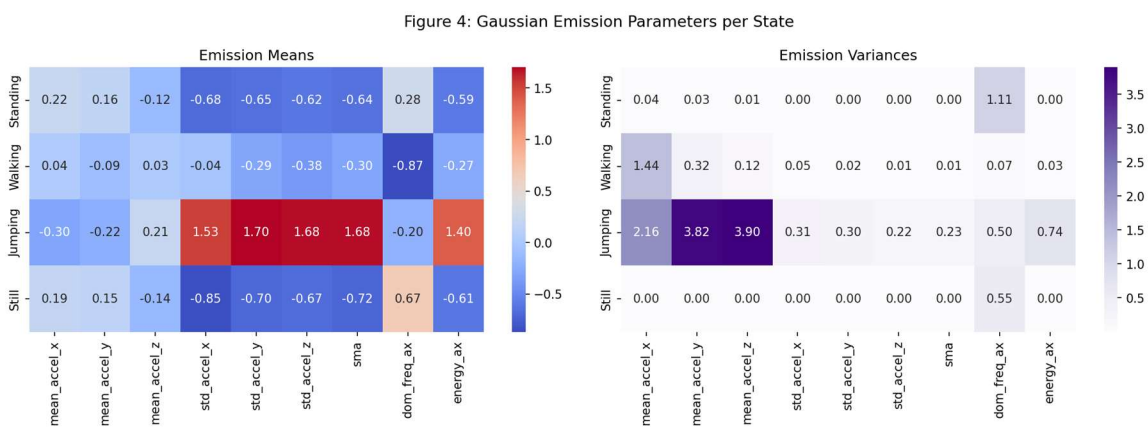

Figure 4: Gaussian emission parameters per state. Left: means. Right: variances. Dynamic activities (Jumping, Walking) separate clearly from static ones (Standing, Still).

4.3 Training Verification

Running Viterbi on the training data (Figure 5) gives perfect classification: every window is assigned to the correct state. This tells us the learned parameters fit the training data well.

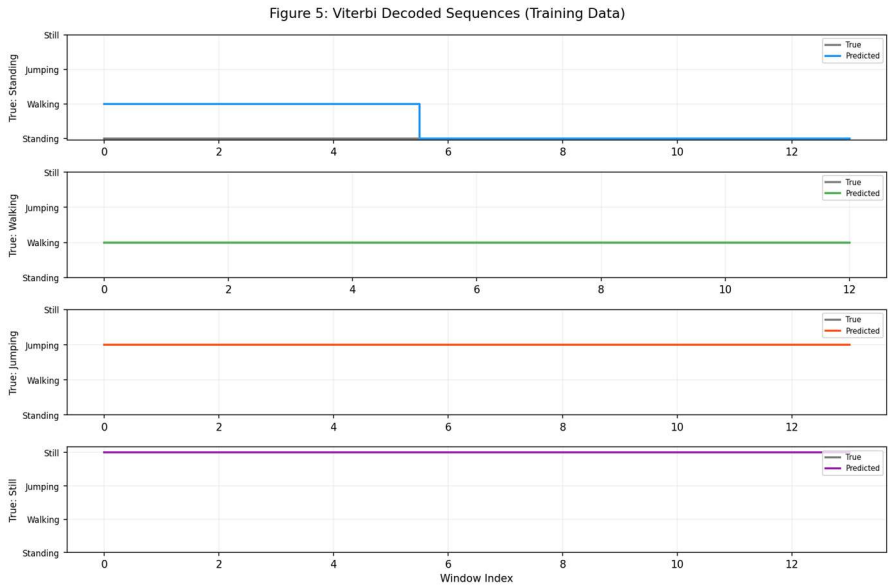

Figure 5: Viterbi decoding on training data. Predicted states (colored) line up exactly with true states (black) for all four activities.

4.4 Evaluation on Unseen Data

We tested the model on two combined multi-activity sequences built from 8 held-out files (2 per activity), all from Phone 2 (Infinix X6855). For each sequence, we concatenated raw sensor data in order (Standing, Walking, Jumping, Still) and then windowed it. This is a harder test than single-activity files because windows right at the transition boundaries contain mixed data from two activities.

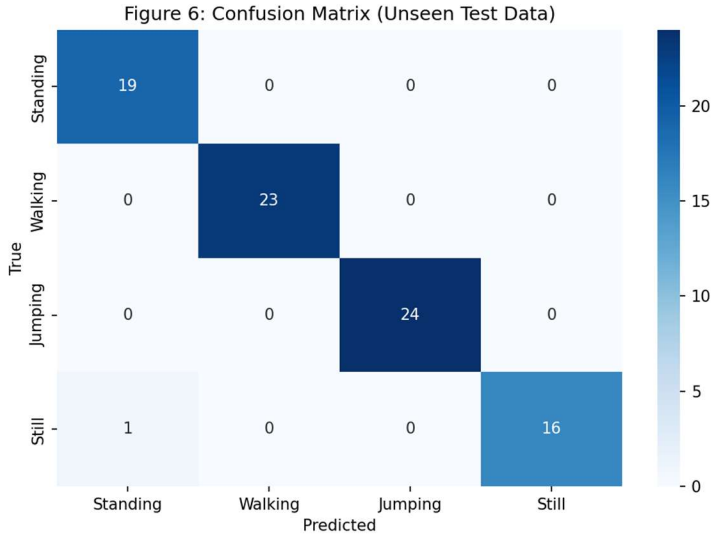

Figure 6: Confusion matrix on test data. Nearly everything falls on the diagonal. The only errors are a small number of Standing/Still confusions.

The evaluation metrics per activity are summarized below:

Activity	Samples	Sensitivity	Specificity	Accuracy
Standing	19	100.00%	98.44%	98.80%
Walking	23	100.00%	100.00%	100.00%
Jumping	24	100.00%	100.00%	100.00%
Still	17	94.12%	100.00%	98.80%
Overall	83	-	-	98.80%

Table 5: Evaluation results on the combined test sequences. Overall accuracy is 98.80%.

Walking and Jumping both hit 100% sensitivity since their signal patterns are quite distinctive. Still had the lowest sensitivity at 94.12%, with one boundary window getting labeled as Standing. Looking at the decoded sequences (Figure 7), the misclassification happens right at the Jumping-to-Still transition, where the window straddles both activities.

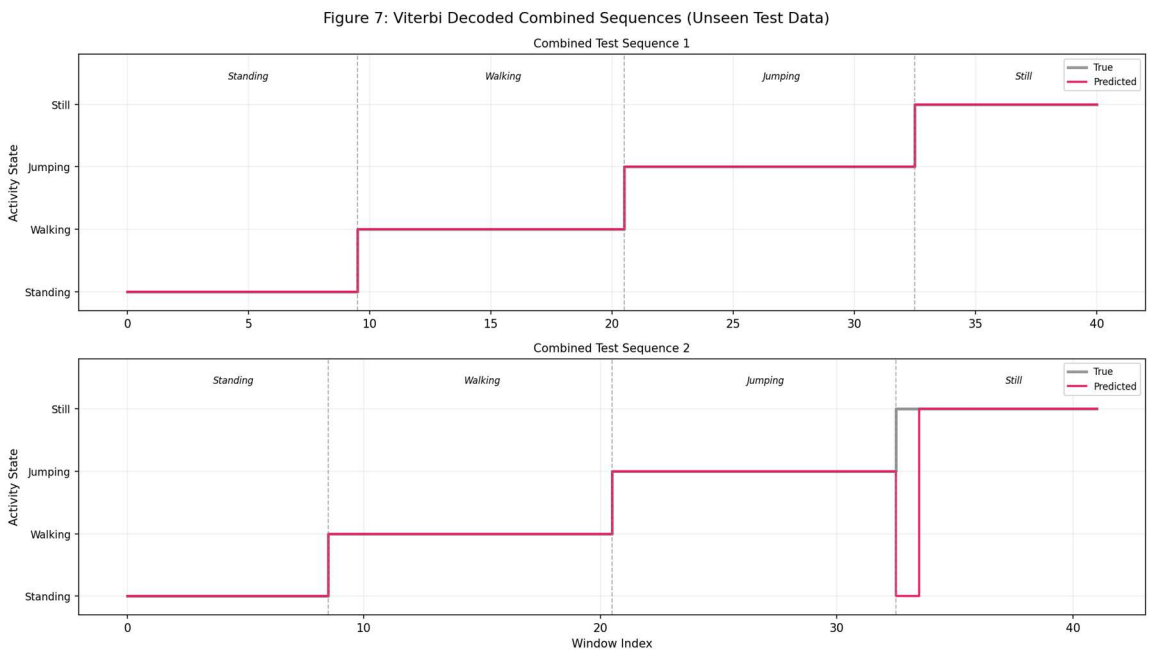

Figure 7: Decoded combined test sequences. Dashed lines show true activity boundaries. Sequence 1 is correct throughout. Sequence 2 has one brief error at the Still boundary.

5. Discussion and Conclusion

5.1 Which Activities Were Easiest/Hardest to Distinguish?

Jumping and Walking were easy to tell apart from everything else. Jumping produces large acceleration variance and spectral energy that nothing else comes close to. Walking has a clear ~2 Hz peak from the step rhythm. Standing versus Still was the hard one. Both are low-movement, and their features overlap quite a bit. The difference comes down to subtle body sway when standing versus essentially no signal when still, and at transition boundaries that difference gets washed out.

5.2 How Transition Probabilities Reflect Realistic Behavior

The high self-transition probabilities (near 1.0) make physical sense. People do not randomly switch activities every second. This also has a practical benefit: the model resists flipping states on one noisy window, which cuts down on false transitions.

5.3 Effect of Sensor Noise and Sampling Rate

Since both phones sample at the same 20ms (50 Hz) rate, we did not need to resample. The two devices do have different sensor hardware, though, so there are slight differences in noise floors and calibration. Z-score normalization went a long way toward smoothing those out. At 50 Hz, we have more than enough resolution for human activities, which mostly happen below 5 Hz.

5.4 Potential Improvements

A few things could make this better. More data from other people would help with cross-person generalization. Features like jerk (the derivative of acceleration) or tilt angle might help separate Standing from Still more reliably. Adding magnetometer data could be useful for orientation-sensitive activities. Training on continuous multi-activity recordings rather than single-activity clips would give the model more realistic transitions to learn from. And using Gaussian mixture emissions (GMM-HMM) instead of single Gaussians per state would let the model handle more complex feature distributions.

5.5 Conclusion

We built a Gaussian HMM from scratch and used it to classify four activities from smartphone sensor data. On unseen multi-activity test sequences, the model gets 98.80% accuracy, correctly identifying all Walking and Jumping windows. The errors are limited to Standing/Still confusion at transition boundaries, which is expected given how similar those two activities look in sensor data. Baum-Welch converged in 6 iterations, and the learned transition matrix lines up with how people actually behave. The results show that HMMs work well for this kind of sequential activity recognition problem, even with a fairly simple model.